Security Review of CashVault Protocol

Audit Conducted by: Nyxaris (Independent Blockchain Security Researcher)

Audit Duration: December 16–23, 2025

Commit Hash: f791ddf Status:

Completed

> **Disclaimer**
>
> A security review is a time-bound effort aimed at providing the highest value possible within the available time. As such, it does not guarantee the absence of vulnerabilities. Additional security reviews, a robust bug bounty program, and continuous on-chain monitoring are highly recommended before any mainnet deployment.

About Nyxaris

Nyxaris is a top-tier independent security researcher with an established track record across major audit platforms.

- Portfolio € CitHub: **Nyxaris Security-Code-Reviews**
- Cantina: Ranked #12 in Infinity Pool competition; active researcher on Alchemix and Badger reviews.
- Code4rena: Consistent contributor with findings in MegaPot, Hybra Finance, and AI Arena.
- Expertise: Deep specialization in DeFi lending, yield optimization, and EVM mechanics.

Overview of CashVault

CashVault is a multi-chain savings protocol designed to foster financial discipline through non-custodial vaults. The protocol enables users to lock assets under specific time-based or goal-oriented conditions.

Core Features:

- **SafeLockVault:** Time-locked savings with an optional "Auto-Relock" feature.
- **TargetedSavingsVault:** Coal-based savings allowing multiple contributors.
- **DCAInvestment:** Accumulation buckets designed for Dollar-Cost Averaging strategies.
- **Fee Engine:** Integrated protocol fee collection (2% standard, 10% emergency).

Scope

The review focused on the following Solidity smart contracts:

- contracts/SafeLockVault.sol
- contracts/TargetedSavingsVault.sol
- contracts/DCAInvestment.sol

Severity Classification Framework

High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Findings Summary

[C-01]	AutoRelock accounting failure leads to fund drainage	Critical	Unfixed
[C-02]	Coal owner can siphon contributor funds via cancel	Critical	Unfixed
[C-03]	Lock duration semantics break extension logic	Critical	Unfixed
[H-01]	Fee-on-transfer tokens cause protocol-wide insolvency	High	Unfixed
[L-01]	Missing zero-address/zero-amount validations	Low	Unfixed
[L-03]	Redundant code and unused structs	Low	Unfixed

Detailed Findings

This security review highlights three Critical vulnerabilities that create a "Perfect Storm" for total protocol insolvency. Below is an expanded technical deep dive into these findings, including the vulnerable code and the corrected implementation.

[C-01] AutoRelock Accounting Failure (Infinite Drain)

The Vulnerability: In SafeLockVault.sol, the claim() function handles the "Auto-Relock" feature. When a user claims, the contract calculates the remaining balance after fees but fails to update the internal position.amount variable. In Solidity, state variables like position.amount persist until explicitly changed.

Vulnerable Code:

```
if (position.autoRelock) {
    position.unlockTime = uint64(block.timestamp) + position.duration;
    // ... fee logic ...
    position.token.safeTransfer(msg.sender, userAmount);
    // ERROR: position.amount is still the original value!
    return;
}
```

Impact: The contract still "believes" the user has the original full amount locked. A user can repeatedly claim their funds until the contract's total liquidity is drained. A user can call claim() every time the unlockTime passes. Since position.amount never hits zero, they can withdraw the same balance repeatedly, essentially stealing other users' deposits stored in the contract.

Recommended Fix:

```
if (position.autoRelock) {
    // 1. Calculate the actual amount to be removed from the vault
    uint256 totalToDeduct = position.amount;

    // 2. Update state BEFORE transfer (Checks-Effects-Interactions)
    position.amount = 0;
    position.active = false; // Or reset logic depending on intended UX

    // 3. Perform transfers
    position.token.safeTransfer(msg.sender, userAmount);
    // ...
}
```

[C-02] TargetedSavingsVault: Contributor Fund Theft

Description: The cancel() function allows a goal owner to terminate a goal and send the entire balance to a destination address of their choice. The contract assumes a "Benevolent Creator" model. It allows the goalOwner to cancel a goal and redirect all funds (including those from other contributors) to any address.

Vulnerable Code:

```
function cancelGoal(uint256 goalId, address payable rescueAddress) external onlyGoalOwner(goalId)
    { Goa
```

Impact: Because this vault supports multiple contributors, a malicious goal creator can wait for others to deposit, then cancel the goal to steal all contributed funds, as there is no refund mechanism for individual contributors.

[C-03] SafeLockVault: Duration semantics broken

Description: The duration variable in the LockPosition struct is overloaded. It is used to validate MAX_DURATION during extensions and as the "period" for auto-relocking.

Impact: If a user extends a 1-month lock by 1 month, duration becomes 2 months. If autoRelock is on, the next lock will inappropriately be for 2 months. Eventually, extensions will cause the lock to exceed MAX_DURATION, making the position unmanageable.

Made with GAMMA